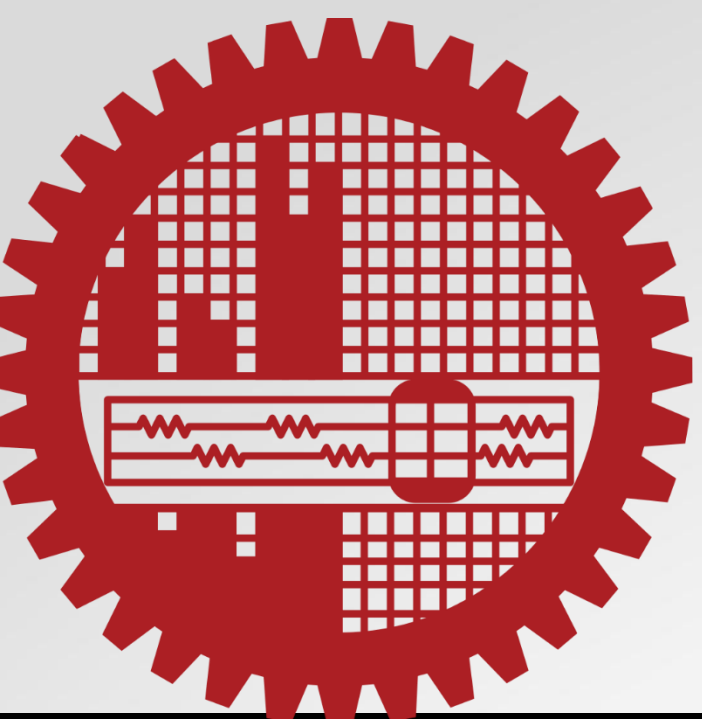




A Prediction Based Replica Selection Strategy for Reducing Tail Latency in Geo-Distributed Systems

Santa Maria Shithil, Muhammad Abdullah Adnan

Bangladesh University of Engineering and Technology (BUET), Dhaka-1000, Bangladesh

Email: shithil.cse01@gmail.com, abduallah.adnan@gmail.com

Background

- Modern applications are built on a geo-distributed system to achieve high availability, reliability, and scalability
- Apart from several advantages, applications deployed on geo-distributed systems suffer from high response time fluctuation that may result in severe degradation on user experiences which in turn reduces the revenue of the service providers
- Among several reasons, tail latency is considered as one of the main reasons for response time fluctuations [1]
- In a geo-distributed system a client's request is split into several subrequests and spread out through the geographically distributed nodes. Long-tail latency occurs when one of the nodes where subrequests are spread out is not responding over other nodes
- Several mechanisms have been developed to reduce tail latency in the geo-distributed systems including request reissuing techniques, request duplication techniques, and replica selection strategies
- Our main goal of this research is to develop a new replica selection strategy to reduce tail latency in a more efficient way and thereby improve the overall performance of the geo-distributed systems

Motivation and Problem Formulation

- A replica selection strategy can efficiently reduce the tail latency in the geo-distributed systems
- Designing an efficient replica selection strategy is very challenging as the performance and loads of the nodes fluctuate over time. Moreover, the latency of a node depends on several factors such as round trip time (RTT), response time which depends on service time and queue size, and the disk input-output operations per second (IOPS) of the nodes.
- It is necessary to keep into consideration all the factors to design a more efficient replica selection strategy to meet the challenges of the geo-distributed systems
- None of the state-of-the-art replica selection strategies considered all the factors and are not efficient enough to reduce the tail latency of the geo-distributed systems.
- For reducing tail latency in the geo-distributed systems we meticulously designed a new prediction-based replica selection strategy that considered all the important factors.
- Our proposed replica selection strategy reduces the tail latency of the system as well as increases the overall throughput of the geo-distributed systems

Proposed Methodology

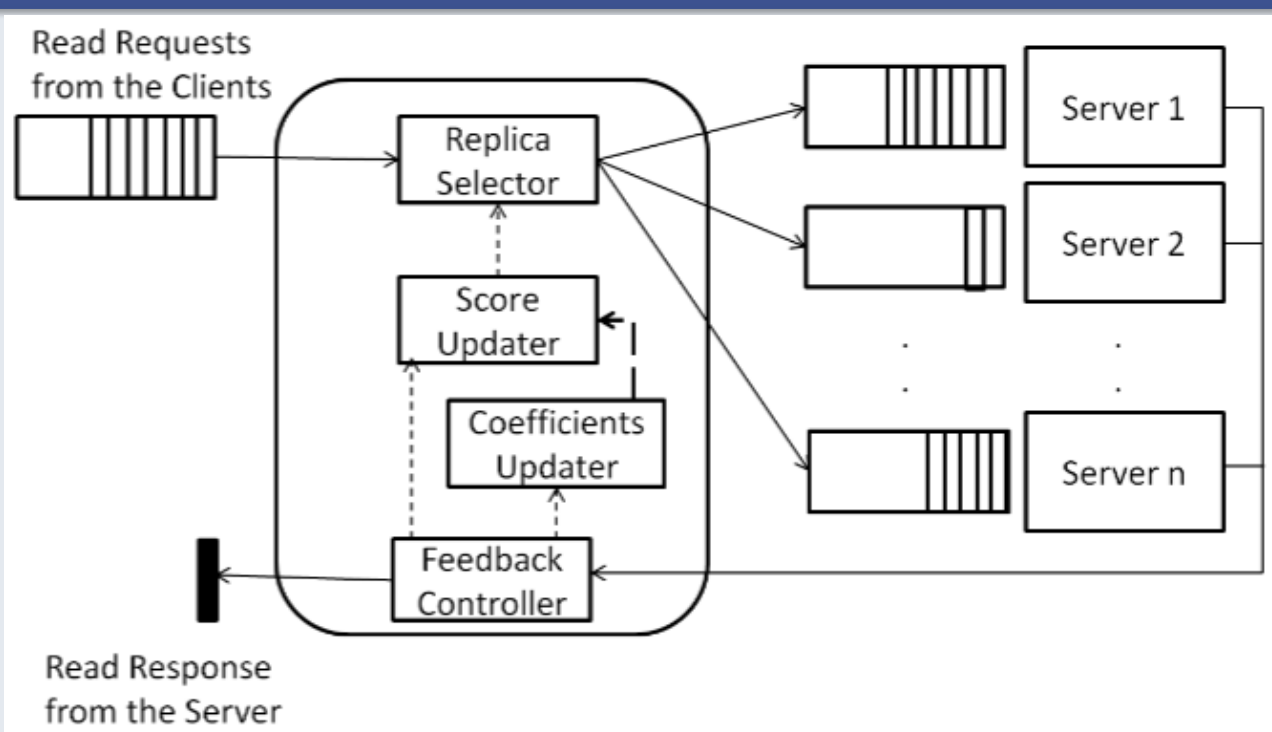


Fig. 1: Proposed system model. Score updater calculates score for each node and replica selector selects the best node based on the score. Feedback controller collect feedback information from the servers and coefficient updater updates coefficients after a specific time interval

- Our proposed strategy works in two steps.
- Step1: predict the latency of all the nodes in a cluster using the following multiple linear regression model where response time and RTT is the dependent variables.

$$L_{ps} = a + b \cdot RTT_s + c \cdot R_s \dots\dots\dots(i)$$

- Where L_{ps} , RTT_s , and R_s are the predicted latency, round trip time, and response time of the node s respectively.
- The coefficients a, b, and c are calculated by using the following equations

$$a = \bar{L}_s - b \cdot \bar{RTT}_s - c \cdot \bar{R}_s \dots\dots\dots(ii)$$

$$b = \frac{\sum R_s^2 \cdot \sum (RTT_s \cdot L_s) - \sum (RTT_s \cdot R_s) \cdot \sum (L_s \cdot R_s)}{\sum RTT_s^2 \cdot \sum R_s^2 - (\sum (RTT_s \cdot R_s))^2} \dots\dots\dots(iii)$$

$$c = \frac{\sum RTT_s^2 \cdot \sum (L_s \cdot R_s) - \sum (RTT_s \cdot R_s) \cdot \sum (RTT_s \cdot L_s)}{\sum RTT_s^2 \cdot \sum R_s^2 - (\sum (RTT_s \cdot R_s))^2} \dots\dots\dots(iv)$$

- To make the prediction more accurate we use Exponentially Weighted Moving Average (EWMA) of both response time and RTT instead of using these directly

- Step2: Calculate a score for each of the node using the following equation and select the node with the lowest score

$$\phi_s = \hat{L}_{ps} + \hat{R}_{ios} \dots\dots\dots(v)$$

- Where ϕ_s , $\hat{L}_{ps}$, and $\hat{R}_{ios}$ are calculated score, EWMA of predicted latency and EWMA of IOPS of each node s.

Experimentation

- To evaluate the performance of our proposed strategy we compare it's performance with dynamic snitch [2], C3 [3], and [4]

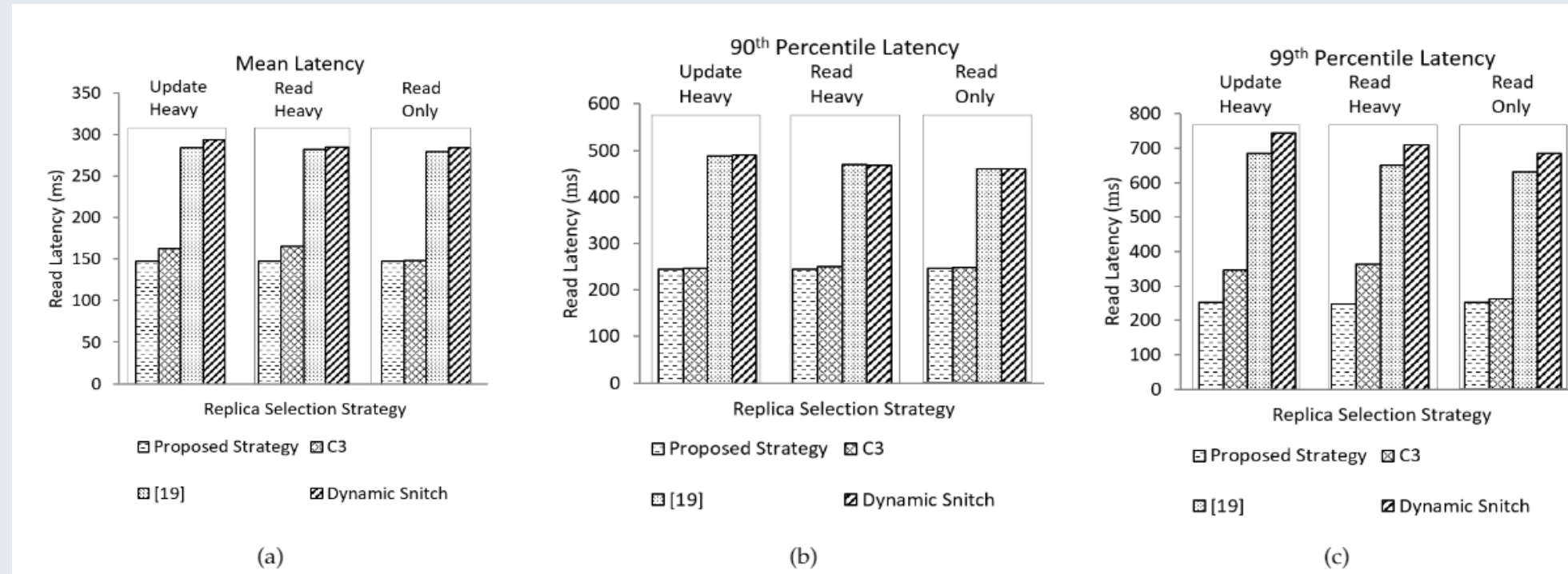


Fig. 2: Latency of Cassandra when using C3, dynamic snitch, [4] and proposed strategy

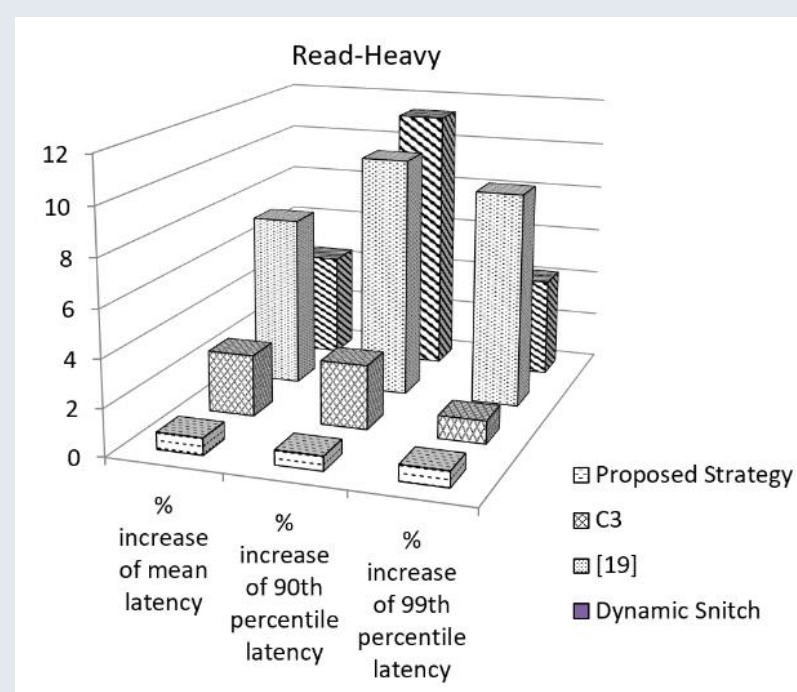


Fig. 3: Percentage increase of mean, 95th and 99th percentile latency of the C3, dynamic snitch, [4] and our proposed strategy with higher workloads

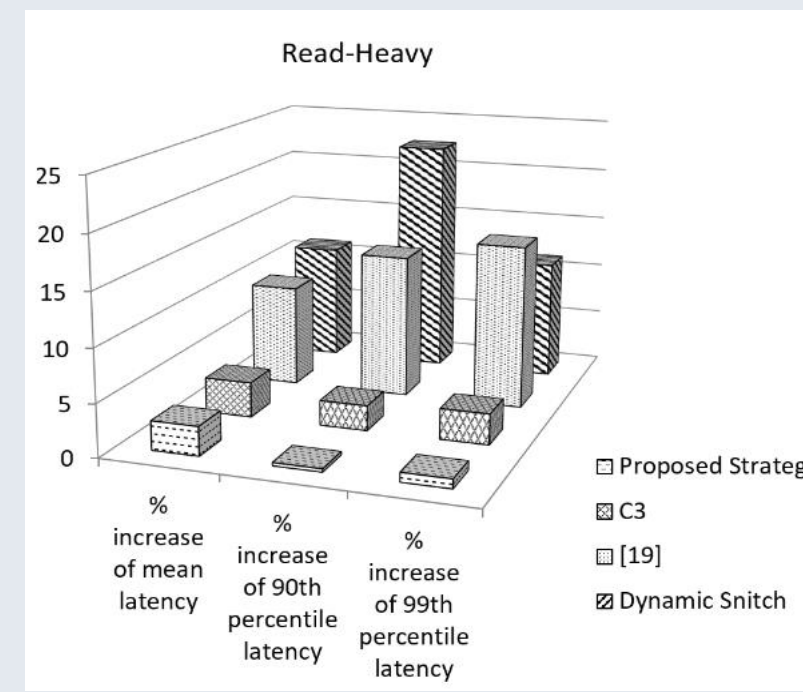


Fig. 4: Percentage increase of mean, 95th and 99th percentile latency of the C3, dynamic snitch, [4] and our proposed strategy at dynamic workloads

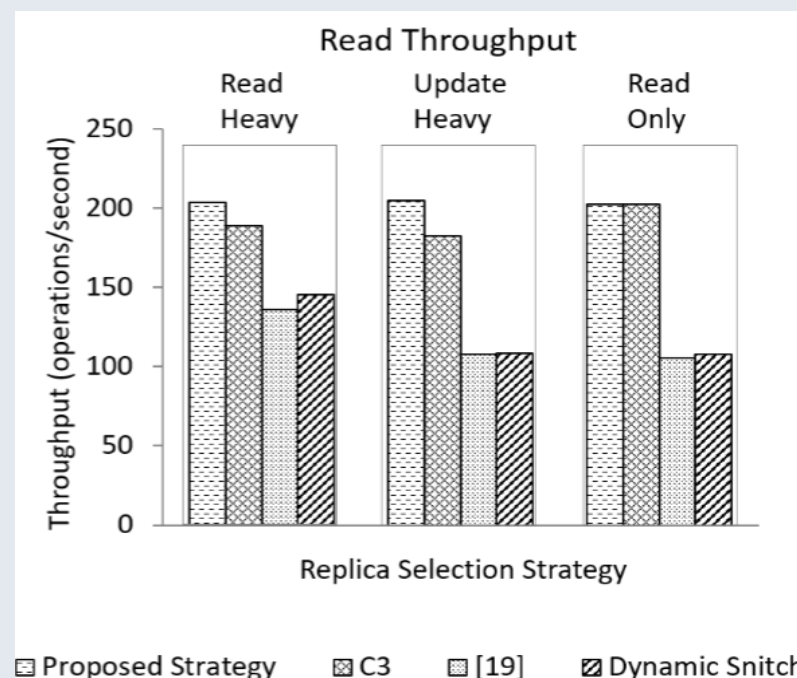


Fig. 5: Comparison of throughput

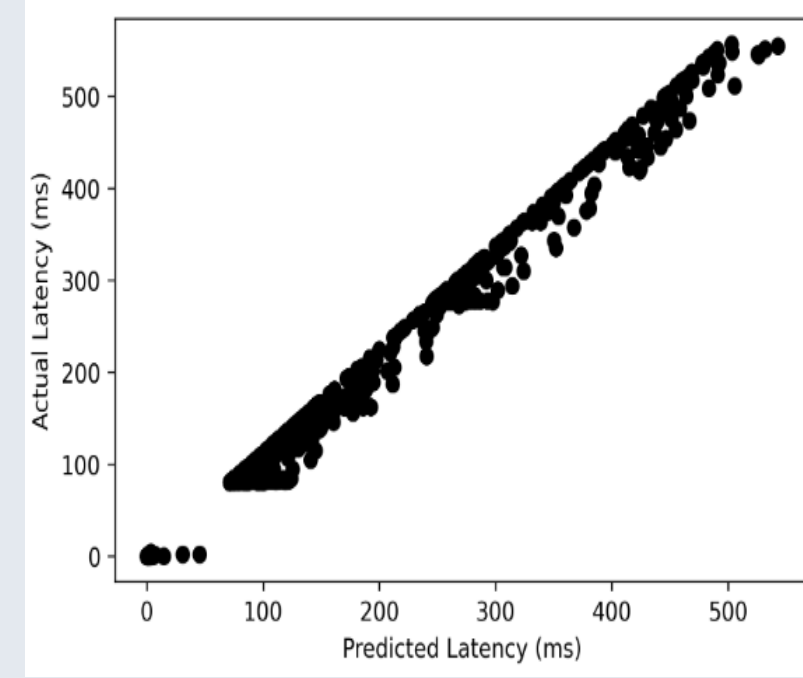


Fig. 6: Predicted latency vs actual latency

Findings

- Considering all the important factors in designing the replica selection strategy helps to reduce the tail latency of the geo-distributed systems
- We can improve the efficiency of the request reissue and request duplication techniques by designing these a top of an efficient replica selection strategy

Conclusion and Future Work

- For evaluating the performance of our proposed strategy, we deployed a 15 node Cassandra cluster on Amazon EC2 that spread across 5 geographical regions
- For generating test datasets and workloads we use industry-standard Yahoo Cloud Serving Benchmark (YCSB) [5]
- Our experimental results show that the proposed strategy not only reduces tail latency but also increases the overall throughput of the systems
- In our proposed strategy the coefficients of equation (i) are recalculated and updated after a user-specified time interval and we did this for coping with the changing situation of the system. An extensive study is required to determine the standard time interval, and, in the future, we will try to find out some standard time intervals for different system settings

References

- [1] Y. Xu et al., Bobtail: Avoiding Long Tails in the Cloud, in Proc. of the Networked Systems Design and Implementation, pp. 329-341, 2013
- [2] A. Lakshman and P. Malik, "Cassandra: a decentralized structured storage system", ACM SIGOPS Operating Systems Review, vol. 44, no. 2, pp. 35-40, 2010. Available: 10.1145/1773912.1773922[Accessed 24 January 2021]
- [3] L. Suresh, M. Canini, S. Schmid, and A. Feldmann., C3: Cutting tail latency in cloud data stores via adaptive replica selection, In Proceedings of the 12th USENIX Conference on Networked Systems Design and Implementation, 2015
- [4] S. M. Shithil and M. A. Adnan, "A Prediction Based Replica Selection Strategy for Reducing Tail Latency in Dis-tributed Systems," 2020 IEEE 13th International Conference on Cloud Computing (CLOUD), Beijing, 2020, pp. 522-526, doi:10.1109/CLOUD49709.2020.00078
- [5] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears, "Benchmarking cloud serving systems with YCSB", In Proceedings of the 1st ACM symposium on Cloud computing, June 10-11, 2010, Indianapolis, Indiana, USA.